

SmartWindow IoT - Sistema Autónomo de Control Climático y Seguridad

Berti Rodrigo Nicolás, Fernandes Leonel Julián, Morandi Mayra Aldana,
Rodríguez Fenske Axel Joel
40742053, 41799417, 41454827, 41107584
Martes, Grupo 2

Universidad Nacional de La Matanza,
Departamento de Ingeniería e Investigaciones Tecnológicas,
Florencio Varela 1903 - San Justo, Argentina

Resumen. SmartWindow IoT es un sistema embebido diseñado para la gestión inteligente de ventanas en entornos domésticos y laborales. Utilizando un microcontrolador ESP32, el sistema integra sensores de lluvia, temperatura, humedad y detección de movimiento (PIR) para automatizar la apertura y cierre de ventanas, mejorando la seguridad y el confort térmico. A diferencia de sistemas puramente reactivos, SmartWindow IoT interpreta el contexto ambiental y las preferencias del usuario mediante una arquitectura de máquina de estados para una operación robusta, incluyendo mecanismos de tolerancia a fallos para asegurar una operación determinista. El sistema incluye control manual remoto vía aplicación Android y un botón de emergencia físico, ofreciendo una solución escalable que permite prevenir daños materiales, optimizar el confort térmico mediante la gestión de ventilación natural y garantizar la seguridad del usuario ante posibles obstrucciones.

Palabras claves: Sistemas embebidos, IoT, SmartWindow, Control ambiental autónomo.

1 Introducción

La gestión del tiempo y la optimización del entorno de operaciones son factores críticos que no pueden dejarse al azar. SmartWindow IoT se presenta como la solución tecnológica indispensable para cualquier entorno que aspire a la excelencia operativa.

El presente trabajo describe el desarrollo de SmartWindow IoT, un sistema embebido basado en el microcontrolador ESP32 capaz de supervisar variables ambientales y actuar de manera autónoma sobre una ventana motorizada. Ante problemáticas frecuentes como el olvido de ventanas abiertas durante precipitaciones, la necesidad de ventilación controlada o la detección de obstáculos durante el movimiento, el sistema utiliza sensores de lluvia, temperatura, humedad y presencia para tomar decisiones de forma automática y segura.

El núcleo del proyecto se fundamenta en la implementación de una máquina de estados determinista, la cual procesa datos provenientes de sensores de lluvia, temperatura, humedad relativa y detección de presencia (PIR) para ejecutar acciones coordinadas sobre un actuador motorizado. Mediante el uso de FreeRTOS, el sistema gestiona la multitarea de manera eficiente, garantizando que la respuesta ante eventos climáticos críticos tenga prioridad sobre las funciones de confort. Este enfoque, más allá de la mera reacción ante estímulos, permite una gestión contextualizada que optimiza la seguridad física, protege los bienes materiales y mejora la eficiencia energética del entorno.

Como parte de la solución desarrollada, se implementó una aplicación móvil Android destinada a proporcionar una interfaz de usuario para la supervisión y el control remoto del sistema SmartWindow IoT. La aplicación permite centralizar la interacción con la ventana inteligente.

La aplicación ofrece funcionalidades de autenticación de usuarios, monitoreo en tiempo real de variables ambientales y visualización del estado actual de la ventana. Además, permite seleccionar el modo de funcionamiento del sistema, alternando entre operación automática y operación manual. En el modo manual, el usuario puede enviar comandos de apertura y cierre directamente al actuador motorizado, mientras que en el modo automático puede supervisar las decisiones tomadas por el sistema a partir de la información proporcionada por los sensores.

Para la comunicación entre la aplicación y el dispositivo embebido se utilizó el protocolo MQTT. Mediante esta arquitectura, la aplicación recibe datos provenientes del ESP32, como temperatura, humedad, estado de la ventana y modo de funcionamiento, y a su vez envía comandos de control cuando es necesario.

De esta manera, SmartWindow IoT constituye una solución integral que reúne tecnologías de sistemas embebidos, Internet de las Cosas (IoT) y desarrollo de aplicaciones móviles.

2 Desarrollo

- Repositorio de GitHub

El repositorio de GitHub puede ser verificado en el siguiente enlace:

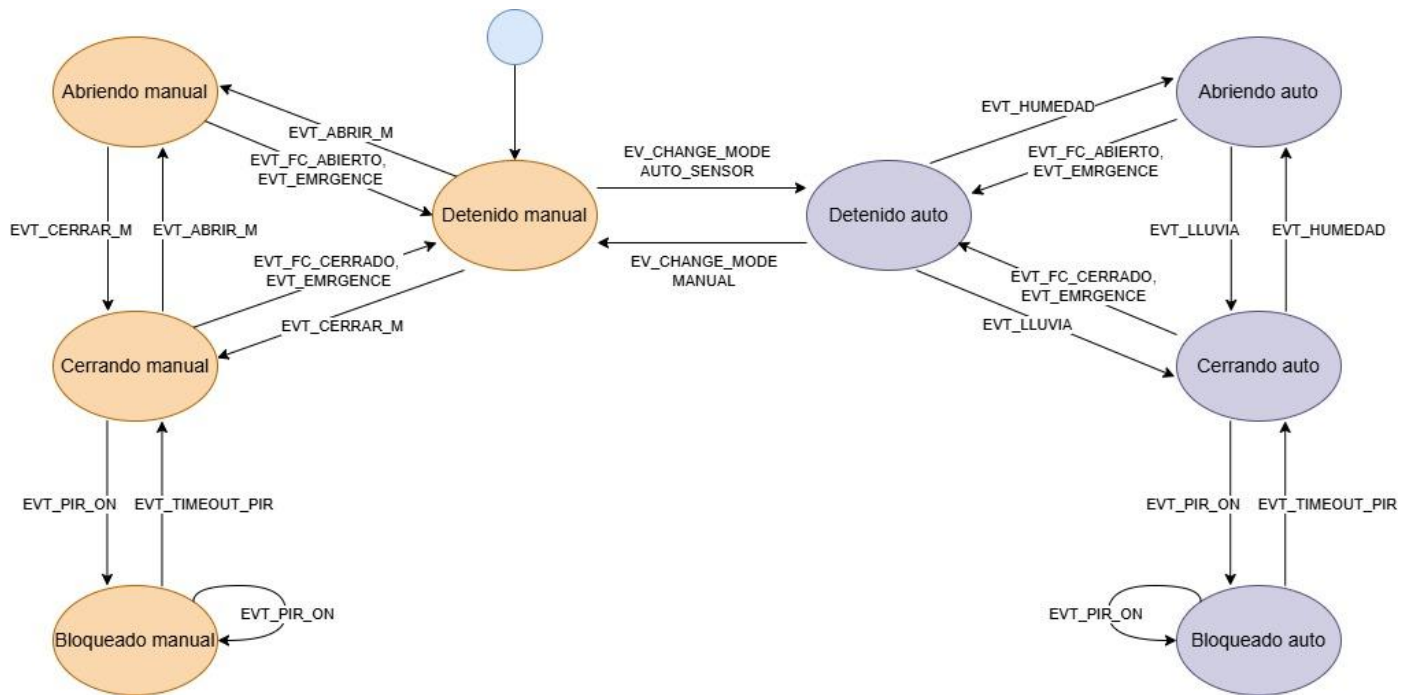
<https://github.com/UNLAM-SOA/2026-SOA-Q1-M2>

- Prototipo funcional

El prototipo funcional puede ser verificado en el siguiente enlace:

<https://wokwi.com/projects/463229357168614401>

- Diagrama de Estados



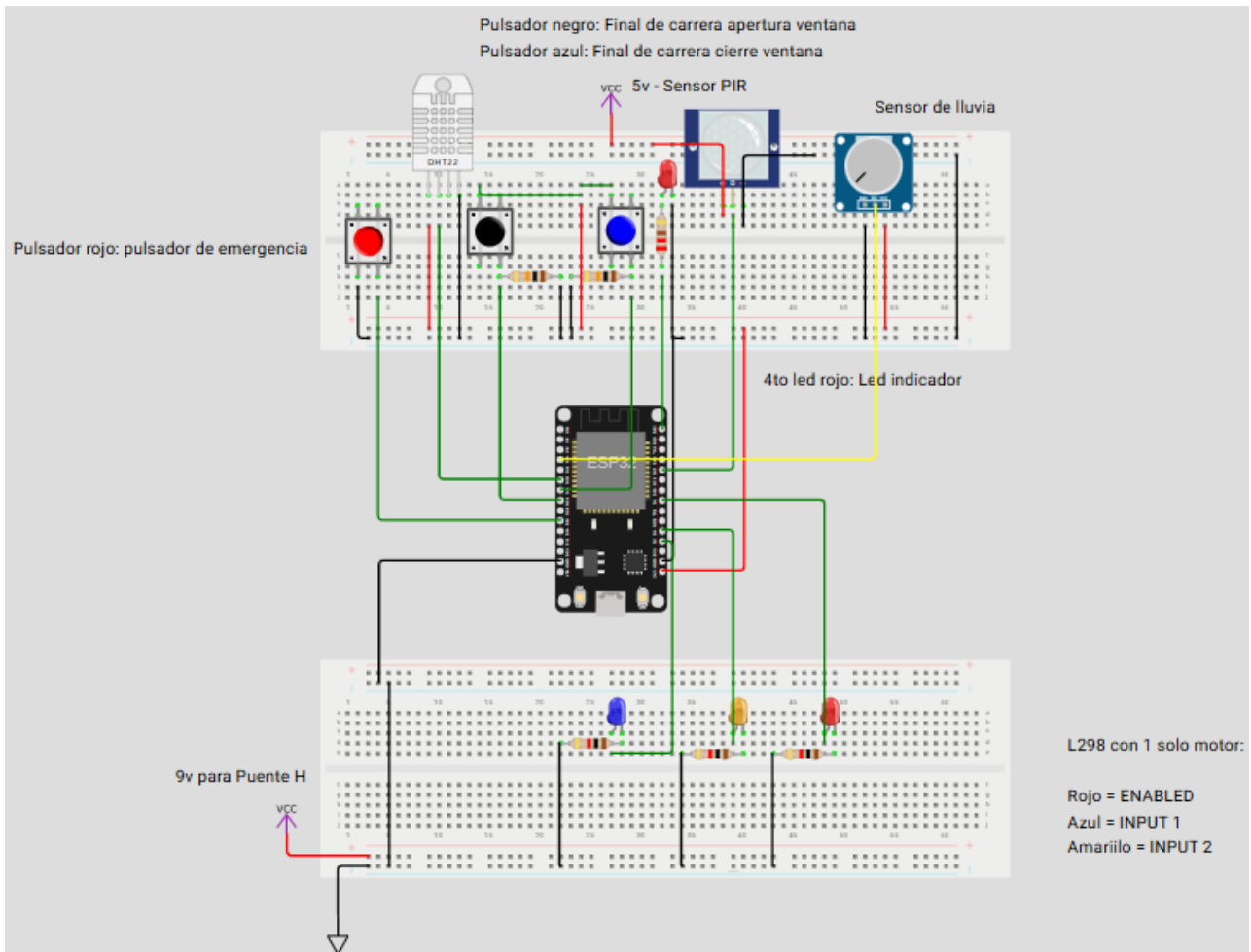
Modo manual

<i>Estado actual</i>	<i>Evento</i>	<i>Próximo estado</i>	<i>Condición</i>	<i>Justificación</i>
Detenido manual	EVT_ABRIR_M	Abriendo manual	Usuario ordena abrir	Inicio de apertura manual
Detenido manual	EVT_CERRAR_M	Cerrando manual	Usuario ordena cerrar	Inicio de cierre manual
Abriendo manual	EVT_FC_ABIERTO	Detenido manual	Fin de carrera abierto	Se completó apertura
Cerrando manual	EVT_FC_CERRADO	Detenido manual	Fin de carrera cerrado	Se completó cierre
Abriendo manual	EVT_CERRAR_M	Cerrando manual	Usuario cambia acción	Cambio de dirección
Cerrando manual	EVT_ABRIR_M	Abriendo manual	Usuario cambia acción	Cambio de dirección
Abriendo manual	EVT_PIR_ON	Bloqueado manual	Detección de movimiento	Seguridad
Cerrando manual	EVT_PIR_ON	Bloqueado manual	Detección de movimiento	Seguridad
Bloqueado manual	EVT_TIMEOUT_PIR	Cerrando manual	Fin del bloqueo	Retoma operación segura
Cualquier estado M	EV_CHANGE_MODE_AUTO_SENSOR	Detenido auto	Cambio de modo	Pasa a automático

Modo automatico

Estado actual	Evento	Próximo estado	Condición	Justificación
Detenido auto	EVT_HUMEDAD	Abriendo auto	Se detecta humedad	Ventilación automática
Detenido auto	EVT_LLUVIA	Cerrando auto	Se detecta lluvia	Protección
Abriendo auto	EVT_FC_ABIERTO	Detenido auto	Fin de carrera abierto	Apertura completa
Cerrando auto	EVT_FC_CERRADO	Detenido auto	Fin de carrera cerrado	Cierre completo
Abriendo auto	EVT_LLUVIA	Cerrando auto	Cambio de condición	Prioriza cerrar
Cerrando auto	EVT_HUMEDAD	Abriendo auto	Cambio de condición	Prioriza abrir
Abriendo auto	EVT_PIR_ON	Bloqueado auto	Movimiento detectado	Seguridad
Cerrando auto	EVT_PIR_ON	Bloqueado auto	Movimiento detectado	Seguridad
Bloqueado auto	EVT_TIMEOUT_PIR	Cerrando auto	Fin del bloqueo	Continúa seguro
Cualquier estado A	EV_CHANGE_MODE_MANUAL	Detenido manual	Cambio de modo	Pasa a manual

- Diagrama de Conexiones

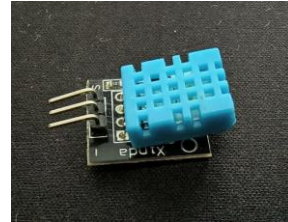


- Funcionamiento de Sensores y Actuadores

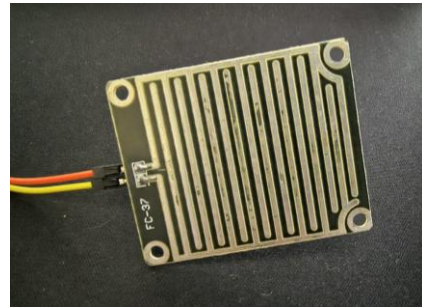
El sistema se compone de los siguientes elementos físicos, los cuales fueron brindados por la catedra:

Sensores:

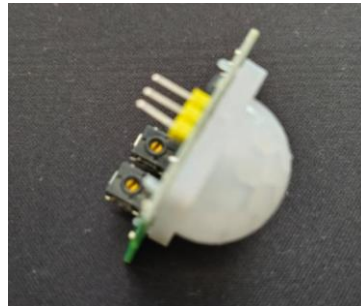
Sensor de Humedad (DHT11): Dispositivo analógico que mide la temperatura y humedad relativa del ambiente para decidir si es necesario abrir la ventana para ventilación.



Sensor de Lluvia: Dispositivo analógico que detecta la presencia de agua, con prioridad alta para cerrar la ventana y prevenir daños.



Sensor PIR: dispositivo electrónico que detecta presencia humana o movimiento para bloquear el motor y evitar accidentes, aumentando la seguridad.

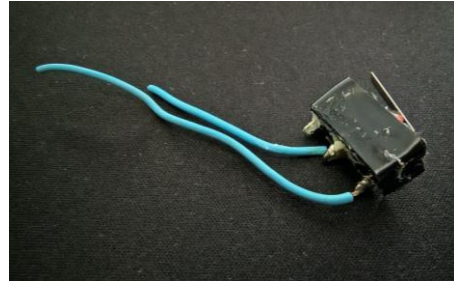


Pulsador: Interruptor eléctrico momentáneo que abre o cierra un circuito solo mientras se mantiene presionado, regresando a su posición original al soltarlo. Pulsador de emergencia.

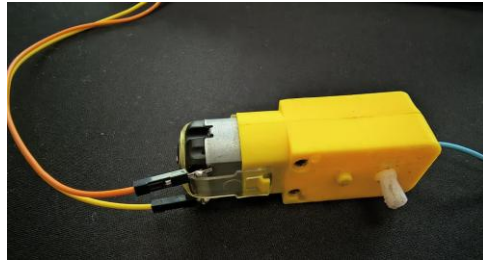


Actuadores:

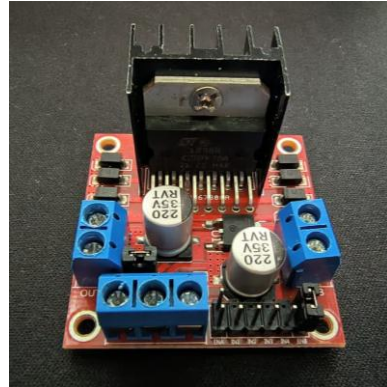
Finales de carrera: Dos dispositivos electromecánicos digitales, que informan al sistema la posición exacta la ventana, si la misma se encuentra abierta o cerrada, permitiendo un control de recorrido preciso.



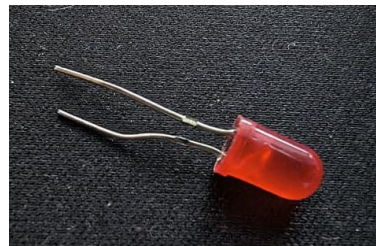
Motor DC: Actuador de corriente continua con escobillas, diseñado para ofrecer movimiento rápido, preciso y controlado en recorridos cortos. Funciona como generador de movimiento mecánico para distintos aparatos.



Puente H (L298N): Circuito electrónico que permite que un motor de corriente continua (CC) gire en ambos sentidos (horario y antihorario) y, controlar su velocidad. El puente H es el intermediario que permite que un cerebro electrónico maneje un actuador que consume mucha energía.

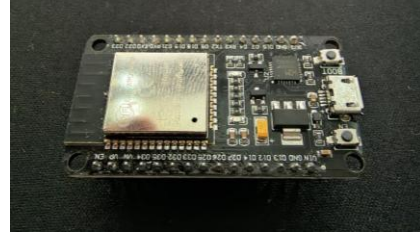


LED (diodo emisor de luz): Componente electrónico que emite luz al pasar corriente en un solo sentido. Se utiliza como indicador visual en circuitos electrónicos.



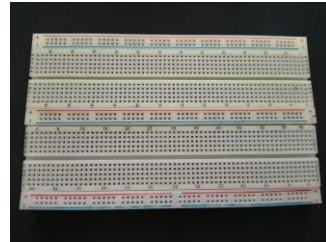
Unidad de control

ESP32: Microcontrolador principal encargado de procesar la lógica, gestionar las tareas mediante FreeRTOS y manejar la comunicación WiFi.

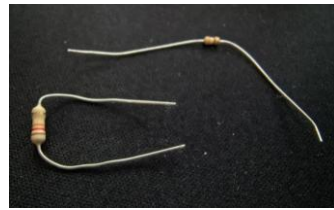


Interconexión y Alimentación

Protoboard: Placa de prototipado que permite armar y probar circuitos electrónicos sin necesidad de soldar, facilitando la conexión de componentes de forma rápida y reutilizable.



Resistencia: Componente electrónico que limita el paso de la corriente eléctrica en un circuito, protegiendo otros componentes y permitiendo controlar valores de voltaje y corriente.



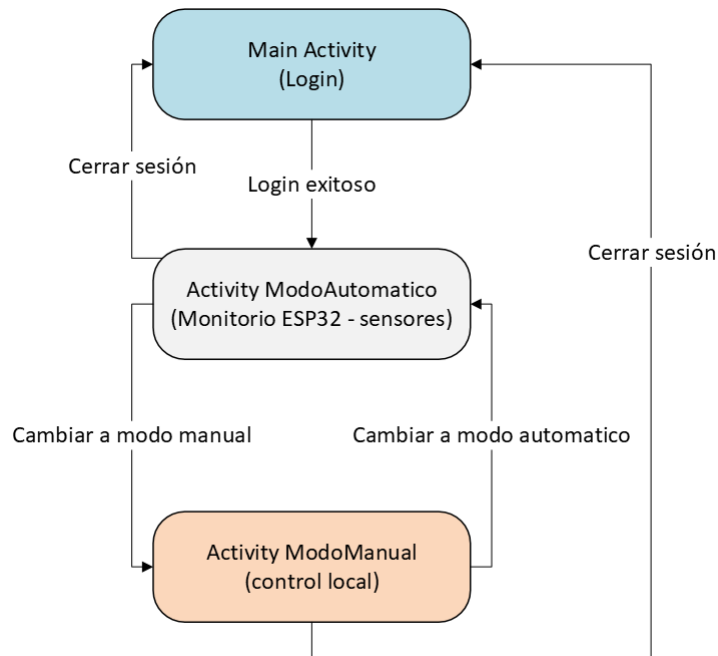
Fuente de alimentación de 9V: Dispositivo que convierte la corriente de la red eléctrica en un voltaje adecuado para alimentar circuitos electrónicos y componentes. En este caso, para el puente H.



Fuente de alimentación de 5V: Dispositivo que convierte la corriente de la red eléctrica en un voltaje adecuado para alimentar circuitos electrónicos y componentes. En este caso, para el Sensor PIR.



- Diagrama de navegación de las Activities



La aplicación se compone de tres Activities principales. La navegación comienza en MainActivity, donde el usuario realiza la autenticación mediante usuario y contraseña. Una vez validado el acceso, la aplicación dirige al usuario a la pantalla ModoAutomatico, desde la cual es posible visualizar el estado de la ventana, la temperatura, la humedad y el modo de funcionamiento actual.

Cuando el usuario selecciona el modo manual, la aplicación navega hacia la Activity ModoManual, permitiendo controlar directamente la apertura y cierre de la ventana mediante comandos enviados al ESP32 a través de MQTT. Desde esta pantalla también es posible regresar al modo automático.

Tanto en ModoAutomatico como en ModoManual existe la opción de cerrar sesión, retornando nuevamente a MainActivity.

- Manual de usuario

El sistema SmartWindow IoT permite supervisar y controlar una ventana inteligente mediante una aplicación Android conectada al dispositivo ESP32. A través de la aplicación, el usuario puede visualizar información del entorno en tiempo real y gestionar el funcionamiento de la ventana de manera remota.

Inicio de sesión

Al iniciar la aplicación, se presenta una pantalla de autenticación donde el usuario debe ingresar sus credenciales. Una vez validado el acceso, se habilita el ingreso a las funcionalidades principales del sistema.

Pantalla principal

Luego de iniciar sesión, la aplicación muestra una pantalla de monitoreo donde se visualizan los datos recibidos desde el ESP32. Entre ellos se encuentran:

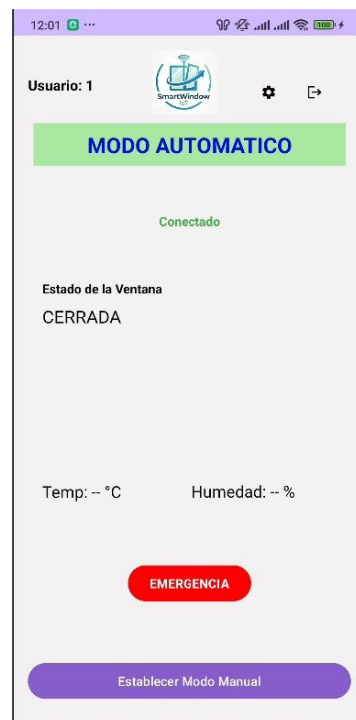
- Temperatura
- Humedad
- Estado actual de la ventana (abierta, cerrada o en movimiento).
- Modo de funcionamiento activo (manual o automático).

La información se actualiza automáticamente mediante la comunicación MQTT establecida entre la aplicación y el sistema embebido.

El sistema permite operar la ventana en dos modos principales:

Modo Manual: El usuario tiene el control total sobre la apertura y el cierre.

Modo Automático: Los sensores deciden la posición de la ventana basándose en factores ambientales (lluvia y humedad).



Modo Automático

El sistema toma decisiones autónomas según la jerarquía de eventos:

Lluvia (Prioridad Alta): Si se detecta lluvia, la ventana se cerrará inmediatamente para proteger el interior.

Humedad (Prioridad Media): Si se detecta exceso de humedad, el sistema abrirá la ventana para favorecer la ventilación.

Si el **sensor PIR** detecta movimiento durante una operación, el sistema bloquea temporalmente el movimiento para evitar accidentes.

Conflicto Lluvia vs. Humedad: En caso de que ocurran ambas simultáneamente, la Lluvia siempre tiene prioridad y la ventana se cerrará.

Modo Manual

El usuario puede cambiar al modo manual desde la aplicación Android. En este modo se habilitan los controles para operar directamente la ventana:

Las acciones disponibles son:

- Abrir ventana
- Cerrar ventana
- Cambiar nuevamente al modo automático

Los comandos enviados desde la aplicación son transmitidos al ESP32 mediante MQTT, quien ejecuta las acciones correspondientes sobre el motor.

Seguridad del sistema

El sistema incorpora mecanismos de seguridad que tienen prioridad sobre las órdenes del usuario.

Si se detecta una situación de riesgo mediante el sensor PIR, el movimiento de la ventana se detiene temporalmente hasta que la condición desaparezca. Asimismo, los finales de carrera garantizan que el motor se detenga automáticamente cuando la ventana alcanza sus posiciones extremas de apertura o cierre.

El sistema también cuenta con un botón de emergencia físico.

En caso de activarse el pulsador de emergencia físico, el sistema interrumpe inmediatamente el movimiento del motor en la posición donde se encuentre. La ventana se detiene por completo y pasa al estado de detenido (manual o automático según el modo en el que estaba operando), quedando fija en esa posición. El sistema no realiza ningún cierre automático posterior ni responde a los sensores climáticos hasta que el usuario envíe una nueva orden manual con el mismo botón, garantizando la seguridad ante cualquier obstrucción.

La aplicación Android incluye un estado de emergencia que se activa con la detección de agitación (shake) mediante el acelerómetro del dispositivo móvil o presionando el botón “EMERGENCIA”.

Al agitar el teléfono o presionar el botón, el sistema procesa la acción de la misma manera que el pulsador físico de emergencia. Esta acción publica de inmediato el evento de emergencia vía MQTT, ordenando al ESP32 detener el motor por completo en su posición actual y congelar la lógica de los sensores climáticos hasta recibir una nueva instrucción manual presionando el botón “DESBLOQUEAR”.

Cierre de sesión

Desde cualquiera de las pantallas operativas, el usuario puede cerrar sesión y regresar a la pantalla de inicio de la aplicación.

Jerarquía de Prioridades

Ante múltiples eventos simultáneos, el sistema responde en este orden estricto:

1. Emergencia / Sensor PIR: Seguridad ante todo (obstáculos o personas).
2. Lluvia: Protección contra daños por agua.
3. Humedad: Optimización del ambiente.

3 Conclusiones

Consideraciones

Para la sincronización de datos en tiempo real entre la interfaz móvil y el sistema embebido, la aplicación Android ejecuta un servicio en segundo plano denominado MqttService. Este componente actúa de forma independiente a las pantallas operativas y utiliza la herramienta LocalBroadcastManager para despachar objetos Intent internos con las actualizaciones de los sensores y estados de emergencia recibidos del ESP32.

Se reconoce como una limitación técnica el uso de LocalBroadcastManager, dado que es una librería declarada obsoleta (deprecated) por Google.

Como alternativa para futuras versiones, la arquitectura de comunicación óptima debe migrar hacia un esquema de persistencia liviano basado en archivos de texto plano mediante SharedPreferences. En este modelo, el servicio en segundo plano sobrescribe un archivo XML interno con los valores actualizados de las variables de control. Al ser texto plano, las pantallas operativas pueden escuchar los cambios en el archivo de manera reactiva, eliminando la creación repetitiva de objetos en memoria, optimizando el uso de batería en el dispositivo móvil y garantizando la persistencia offline del último estado conocido de la ventana.

Las consultas y operaciones de escritura en la base de datos local (implementada con la librería Room) se delegaron a hilos (threads) en segundo plano. Esto se debe a que la arquitectura de Android prohíbe de manera estricta ejecutar operaciones de entrada/salida en el hilo principal, previniendo así el congelamiento de la interfaz de usuario (ANR).

Lecciones Aprendidas

Como principal lección aprendida en el diseño del programa, se reconoció una deuda técnica en la estructura de la máquina de estados frente a las situaciones de emergencia (activadas por el pulsador físico, el botón digital o el shake de la app). Desde la teoría

de sistemas embebidos, estas alertas críticas debieron gestionarse mediante un estado exclusivo de "Bloqueo por Emergencia", el cual aislara por completo al sistema y obligara a pasar por una transición única.

En la arquitectura actual, los eventos de emergencia son recibidos y escuchados por los mismos estados activos que atienden al sensor PIR. Esto genera una vulnerabilidad de diseño, ya que hasta que no finalice el procesamiento del evento del PIR, la FSM no puede detectar ni procesar la emergencia.

Debido a que esta necesidad se detectó en una etapa avanzada del proyecto, la baja disponibilidad de tiempo impidió reestructurar por completo la FSM. Como solución de contingencia, se optó por forzar el retorno a los estados Detenido preexistentes e interrumpir el Puente H por software. Aunque esta implementación resolvió el problema operativo con éxito, el análisis evidenció la importancia de planificar los estados desde las fases iniciales del diseño para lograr un modelo formalmente óptimo.

El servicio MqttService fue diseñado e implementado con la arquitectura de un servicio en primer plano: Foreground Service.

El objetivo inicial de esta decisión de diseño fue permitir el envío de notificaciones críticas en tiempo real (como alertas de lluvia o intrusión por el sensor PIR) que impacten en el dispositivo móvil incluso si este se encuentra bloqueado o con la aplicación minimizada.

Debido a las restricciones de tiempo en la etapa de desarrollo, la interfaz de notificaciones push en la barra de estado no llegó a completarse, por lo que el componente opera temporalmente en segundo plano (background), procesando los datos de forma invisible para el usuario. Se establece la implementación de las alertas visuales como una mejora para una futura versión del sistema.

De cara a próximas iteraciones, se plantea como una línea de mejora el rediseño estético de la aplicación móvil. Si bien la interfaz actual cumple los requisitos operativos de control manual y monitoreo automático, resulta conveniente migrar hacia un diseño visualmente más moderno, amigable y agradable a la vista.

4 Referencias

1. SOA-UNLaM. Disponible en: <http://www.soa-unlam.com.ar/wiki/index.php/PUBLICO:Portada>
2. SOA UNLaM: PUBLICO: Sensor de humedad. Disponible en: https://www.soa-unlam.com.ar/wiki/index.php/PUBLICO:Sensor_de_humedad
3. Wokwi: ESP32 DHT11 simulation. Disponible en: <https://wokwi.com/projects/397229975334287361>
4. Wokwi: Sensor PIR simulation. Disponible en: <https://wokwi.com/projects/426323956437045249>